One Program, Many Circuits: Backend-Aware Circuit Optimization for Private AI

(author names elided for review)

Abstract

Privacy-preserving machine learning is written by experts, by hand, against a chosen cryptographic backend. We argue it should be compiled, and that a compiler can do better than the expert — not because it is cleverer, but because batching independent work is a property of a schedule rather than of an author's attention. We present Baru, a language whose programs compile to a single circuit IR and are then optimized *differently depending on which privacy backend they target*. On a neural network with secret weights, compiled code issues 0.38× the interactive protocol invocations and 0.49× the messages of a hand-written, hand-batched expert baseline, while computing a bit-identical result; the source says nothing about batching. We report an ablation in which each compiler pass acts on a different axis — scheduling collapses rounds 285×, common subexpression elimination collapses circuit size 68× and bandwidth 30×, and rebalancing dependent reductions turns a fold's $2n-1$ rounds into $2\lceil \log_2 n \rceil + 1$. Finally we show the property that motivates the design: the *same* rewrite is a 2.24× win under one protocol and worth exactly nothing under another, so a single optimized circuit cannot serve both. Optimizing one program per backend yields up to three distinct circuits, and running a circuit under the backend it was not optimized for costs up to 2.33×.

1 Introduction

Systems for private machine learning — secure multi-party computation (MPC), homomorphic encryption (FHE), and hybrids of the two — are overwhelmingly written by hand. The reason is not ignorance of compilers; it is that the performance of these systems depends on details a general-purpose compiler is not aware of. Under a secret-sharing protocol the dominant online cost is not work but *communication rounds*, and the difference between a layer that costs one round and one that costs n is invisible to the source text.

This paper starts from an observation about our own code.

Experts serialize layers, and do not notice. Our hand-written private-inference pipeline contained a scheme-switch import that issued three blocking communication rounds *per activation* — 16 224 sequential round trips for a single convolution. It accounted for 88% of end-to-end runtime. It survived review, survived benchmarking, and a draft of a companion paper had already concluded in print that the bottleneck lay elsewhere. It was found only by per-phase instrumentation.

That was not an isolated slip. In the same codebase, the composable gadget library — the API a user is meant to write against — issued one comparison and one multiplication *per element* for ReLU, and one multiplication *per weight* for a dense layer. Measured over a real transport on a 64-wide layer, this cost 5–14× the messages and 9–45× the wall-clock of the same computation issued through batched primitives, with byte counts within 5% and outputs bit-identical. The defect is invisible to a security argument, invisible to a correctness test, and invisible at zero latency. Only a round count exposes it.

The claim. A compiler does not have this failure mode. An as-soon-as-possible schedule over the dependence graph batches *everything* that is independent, uniformly, without being asked. We show that compiled code therefore matches and then beats expert hand-written code on a real network, and — the part we believe is new — that the right circuit *depends on the backend*, so a compiler that optimizes once and emits one circuit is leaving a measurable amount on the table.

Contributions.

1. **Parity, then advantage (Section 4.2).** Compiled code matches hand-batched code message for message at the layer level, and on a whole network with secret weights issues 0.38× the interactive protocol invocations and 0.49× the messages of the expert baseline.

2. **An ablation with separable axes (Section 4.3).** Each pass contributes on a different metric: scheduling 285× on rounds, CSE 68× on circuit size and 30× on bandwidth, reduction rebalancing 1.25× on rounds. None substitutes for another.

3. **Rebalancing *compound* reductions (Section 3.1).** Classical reassociation handles chains of a single associative primitive. The reductions an AI workload contains are not primitives — a maximum is $\mathrm{acc} + (x > \mathrm{acc}) \cdot (x - \mathrm{acc})$, four instructions with two entry wires. We recognise an associative reduction by repeated subgraph shape, verify associativity by evaluation, and rebalance: $2n - 1$ rounds become $2\lceil \log_2 n \rceil + 1$.

4. **Backend-aware optimization, measured (Section 4.5).** We optimize one program once per backend and price every resulting circuit under every backend. The diagonal wins every column; the off-diagonal entries are what a backend-blind compiler loses.

2 Background and setting

Rounds, not operations. In secret-sharing MPC, local arithmetic is free and every multiplication or comparison requires interaction. Because independent interactive operations can be merged into one round, the cost of a program is governed by the *depth* of its dependence graph, not its size. A layer of 512 independent multiplications costs one round; a chain of 512 dependent ones costs 512.

Backends differ in what they charge for. An arithmetic secret-sharing protocol pays per round and treats addition as free. A binary protocol pays per AND gate and has real depth inside an adder. A constant-round garbled protocol pays for gates but not for depth at all. A rewrite that trades depth for nothing is therefore a large win for the first, a moderate one for the second, and worthless for the third — from one rule.

Threat model and scope. All results are semi-honest. Protocol availability follows the deployment (party count, majority assumption); the comparisons in Section 4 use the protocols named in each table. Our executed measurements use a three-party honest-majority replicated protocol [8]; the analytic cost model spans a wider registry and is the basis of Section 4.5. Section 6 states which numbers are measured and which are modelled.

3 Baru

Baru is a small expression language. A program is compiled to a static single-assignment circuit IR (`FlatOp`) shared by every backend, then lowered to a per-protocol instruction program, scheduled, and either costed or executed.

$$\text{Expr} \;\to\; \text{FlatOp (SSA)} \;\to\; \text{lower} \;\to\; \text{schedule} \;\to\; \{\text{cost},\ \text{execute}\}$$

Scheduling is where batching comes from. The schedule is ASAP over the dependence DAG: a local instruction inherits the level of its latest operand; an interactive instruction sits one level later, and every interactive instruction at a level executes in one round. ASAP is optimal for depth, and depth is the round count, so this is the minimum-round schedule *for a fixed lowering*. Crucially, no vectorized IR is required: independent scalar operations land at the same level and are issued through one batched primitive.

Optimization is search plus a cost model. Rewrites are proposed per site and accepted when the backend's cost model says they pay. The rules used here are algebraic identities, common subexpression elimination, shift fusion, strength reduction, classical reassociation, and reduction rebalancing (Section 3.1). Rules whose sign depends on the operand are searched site by site, so a rewrite can be taken at one site and declined at another within the same circuit.

3.1 Rebalancing compound reductions

Classical reassociation rebuilds $((a \cdot b) \cdot c) \cdot d$ into $(a \cdot b) \cdot (c \cdot d)$ for a single associative operator. AI reductions are not single operators. A maximum-fold step is

$$\text{acc}' \;=\; \text{acc} + (x > \text{acc}) \cdot (x - \text{acc}),$$

four instructions forming a subgraph with two entry wires. Our rule peels a *step region* — the instructions between the accumulator wire and the step's result — requiring that every instruction inside is private to the region and that exactly two wires enter it. Steps are canonicalised to a shape key; a chain of equal-keyed steps is a reduction, and its leaves are rebalanced into a tournament preserving left-to-right order, so the rewrite needs associativity and never commutativity.

Associativity is tested, not assumed. The search accepts candidates on *cost*, not on semantics, so a rule cannot rely on it to reject an unsound rewrite. Before offering the rewrite we evaluate the step operator on edge values and random triples, checking $f(f(a,b),c) = f(a,f(b,c))$; a subtraction fold is correctly left alone. The optimizer additionally differentially checks the whole resulting circuit.

Three implementation subtleties determined whether the rule worked at all, and we record them because each produced a plausible wrong answer rather than a failure. (i) The accumulator is read three times *inside its own step*, so "used once" is the wrong privacy test; the correct condition is that every use lies inside the step that consumes it. (ii) A fold's inner roots are also valid chains, so taking the first match rebalances a prefix and leaves the tail serial. (iii) A rebalanced tree still *contains* chains — every root-to-leaf path is one — so a fixpoint pass will happily rebuild a balanced path into a deeper one; the rule must refuse any rewrite that does not strictly reduce depth.

Table 1: Round count is flat in layer width. No vectorized IR, no annotation.

program	n	interactive ops	rounds
`relu-layer`	1	3	3
`relu-layer`	64	129	3
`relu-layer`	512	1025	3
`dot`	512	513	2

4 Evaluation

Every figure is produced by a command in the artifact, and every route in every comparison is checked against a plaintext integer reference — a cost comparison between routes computing different things is meaningless, and our harnesses fail rather than report one. Wall-clock is single-threaded on commodity hardware; ratios transfer, absolute times do not.

4.1 Scheduling batches without being asked

Table 1 compiles layer-shaped programs and reports the ASAP round count. Interactive operations grow by more than two orders of magnitude while the round count does not move.

4.2 A whole network against an expert

The network is $x(n) \to \text{dense}(h) \to \text{ReLU} \to \text{dense}(c) \to \max$, with *secret weights* throughout, so every multiplication is a real interactive operation. Two routes evaluate it over the same three-party transport:

- **gadget** — hand-written code calling a batched gadget library. This is the honest expert baseline: those gadgets match hand-tuned code message for message.
- **compiled** — the same network as a Baru expression, through the circuit compiler and a level-batched executor. The source is scalar arithmetic, a mux-style ReLU and a left-fold maximum. It says nothing about batching.

At n=32, h=16, c=8, RTT 20 ms: the expert route issues 39 interactive protocol invocations and 95 messages; the fully optimized compiled route issues **15** and **47** — 0.38× and 0.49× — at parity on wall-clock. The advantage *grows with width*: the compiled route stays at 15 invocations while the expert route grows from 31 at n=16 to 39 at n=32.

Two things we do not claim. Without reduction rebalancing the compiled route *loses* (1.88× wall-clock), because the expert's maximum is a hand-written tournament; that ablation is reported rather than hidden. And the wall-clock is parity, not a win: both routes perform the same number of comparison layers, which dominate latency under a secret-sharing protocol. The compiler's gain is in multiplications and sharing.

4.3 What each pass contributes

Table 2 adds one pass per row, starting from an executor written the obvious way: walk the circuit, call the backend per operation.

Table 2: Ablation, n=8, h=4, c=4, zero latency. Every row computes the same value.

route	ops	interactive	messages	bytes
`gadget` (expert)	—	20	53	56 k
`compiled: serial`	7419	4280	5859	1798 k
+ ASAP batching	7419	15	47	1682 k
+ CSE	109	15	47	55.7 k
+ reassociation	109	**12**	**37**	55.5 k

Table 3: Cross-application, `max-fold`, n=8, WAN, three parties. Predicted milliseconds; $*$ marks each circuit under the backend it was optimized for. Every circuit was differentially verified.

circuit optimized for	as `shamir`	as `semi-bin`	as `bmr`	rules kept
`shamir`	1453*	3761	282	reassociate-reduction
`semi-bin`	1453	3761*	282	reassociate-reduction
`bmr`	3253	8761	282*	—

The passes act on *different axes*: scheduling is 285× on interactive invocations and 1.00× on circuit size; CSE is 68× on circuit size and 30× on bytes while changing no round count; reassociation is 1.25× on rounds and nothing else. A single-column summary would report CSE as a 1.00× no-op, which is why we report the ladder.

The `serial` row deserves emphasis. It is 285× the interactive invocations of the *same circuit* under ASAP levelling, on essentially identical bytes. The round-structure defect that our human experts shipped (Section 1) reappears inside a naively written executor, and is removed by scheduling alone.

4.4 Circuit quality on a real workload

Common subexpression elimination is a textbook pass, but its absence from the path that feeds a backend is easy to miss: in our system it existed only behind an explicit optimizer entry point, so every circuit handed to the FHE and ZKP backends carried whatever redundancy the source contained. A fold written the obvious way repeats its accumulator subtree at each step and flattens *exponentially*: 4372 operations at n=8, collapsing to 28. On a real trained image classifier the same pass takes the emitted circuit from 2365 to 1764 gates (−25%) with bit-identical outputs.

4.5 The same program, a different circuit per backend

Divergence on its own proves nothing: two circuits can differ in ways nobody pays for. We therefore optimize one program once per backend and price *every* resulting circuit under *every* backend (Table 3).

The diagonal wins every column. The mechanism is visible in the last column: the *same* rewrite is kept for the two interactive protocols and declined for the constant-round garbled one, because depth is the round count for the former and free for the latter. Running the garbled backend's circuit under `shamir` costs 2.24×. Nothing told the search this; the cost model priced it.

Across three workloads — an independent layer, a dependent reduction, and a dot product against public constants — one program yields 2, 2 and 3 distinct circuits, with worst wrong-circuit

penalties of 1.66×, 2.33× and 1.81×. Where a backend keeps no rules it genuinely wanted the original circuit; an optimizer that always rewrites is not being selective.

Protocol assignment. The same cost model assigns operation kinds to protocols. On a dense layer with a comparison it returns cmp-lt → binary with multiplications left arithmetic — the classical hybrid-MPC split, derived rather than annotated — worth 2.03× over the better pure protocol on a 32-wide layer.

5 Related work

Protocol assignment. HyCC [2], Silph [3] and SING [1] assign MPC protocols to parts of a program and pay for the conversions. Silph solves the assignment with integer linear programming [3], which forces a linear cost model; SING [1] replaces the solver with a graph neural network, reporting large speedups at comparable quality and supporting non-linear costs. Our objective is non-separable for two structural reasons — a dependence level costs the *maximum* over its members, and conversions batch per level — so an ILP formulation does not apply, and the exact model is cheap enough to evaluate that a learned surrogate is not yet needed. We assign per operation *kind* rather than per instruction: a coarser space, but an interpretable answer. *These systems are MPC-only.*

Circuit compiler infrastructure. CirC [9] shares our one-IR-many-backends structure, with frontends and backends over a typed term IR, and includes oblivious and linear-scan array elimination. Its independent arrival at the same architecture is reassuring. CirC optimizes for a target but does not, to our knowledge, report circuits that *diverge* per backend under a shared cost model.

Hand-built private inference. Gazelle [5], Delphi [7], Cheetah [4] and CrypTFlow2 [10] are hand-designed systems that split linear layers and nonlinearities across FHE and MPC. They are the expert artefacts this paper argues a compiler should be able to derive; our assignment reproduces the shape of that split from a cost model, though see Section 6.

MPC compilers. MP-SPDZ [6] optimizes rounds by merging independent openings — the same insight our scheduler implements — and is blind to FHE. FHE compilers optimize multiplicative depth and are blind to MPC. Neither can ask whether a computation should be an FHE program at all, because neither has a cost model spanning the alternatives.

6 Limitations

Measured versus modelled. Section 4.2 and Section 4.3 are executed: real serialization, real message passing, real byte counting, verified against a plaintext reference. Table 3 and the assignment results are *predictions* from the analytic cost model. The model's online round and volume terms are exact for the lowering; its offline volume and local-work constants are literature estimates, and we do not quote them as measurements.

Metric. "Interactive invocations" counts batched protocol calls at the backend interface, which is exactly comparable between routes but is a proxy for rounds rather than a round count: one batched comparison runs several rounds internally.

Scope of the compiled path. The compiled network is MPC. Our FHE/MPC hybrid remains orchestration outside the language — the language does not express scheme switching — so the expert baseline in Section 4.2 is the batched gadget library rather than the more heavily tuned hybrid pipeline. Extending the language to express switching is future work and would strengthen the backend-aware claim, since the compiler could then choose where to switch.

Assignment with FHE. A previously recorded result placed multiplications on FHE for wide layers. It did *not* reproduce in the configuration we re-ran for this paper, which returned the binary comparison split only. We therefore do not claim it. **[TODO: re-derive or retract; do not cite the earlier number]**

Workload scale. The compiled networks here are small MLPs. Convolutional models at image scale are compiled and costed but not executed end-to-end through the compiled path. **[TODO: scale the compiled path to the convolutional model, or state the ceiling analytically]**

Optimality. The rewrite search is greedy and site-independent; a pair of rewrites that only pays jointly is missed. Assignment is greedy over operation kinds and optimal assignment is NP-hard.

7 Conclusion

Private machine learning is written by hand against a chosen backend, and the choice is usually made before the program is. We have argued that both decisions should be compiled: that a schedule batches independent work more reliably than an author does, and that the circuit worth emitting depends on the backend it is headed for. The evidence for the first is that compiled code beat our own experts on their own network, and that the defect it removes is one those experts shipped three times in one codebase. The evidence for the second is that a single rewrite is a 2.24× win under one protocol and declined under another, from one rule and one cost model.

References

[1] Blüml, Huppert, Khayata, Schmidt, and Schneider. SING: Hybrid MPC protocol assignment with graph neural networks. Cryptology ePrint Archive, Paper 2026/1237; USENIX Security 2026, 2026. `https://eprint.iacr.org/2026/1237`.

[2] Niklas Büscher, Daniel Demmler, Stefan Katzenbeisser, David Kretzmer, and Thomas Schneider. HyCC: Compilation of hybrid protocols for practical secure computation. In *ACM CCS*, 2018.

[3] Edward Chen, Jinhao Zhu, Alex Ozdemir, Riad S. Wahby, Fraser Brown, and Wenting Zheng. Silph: A framework for scalable and accurate generation of hybrid MPC protocols. In *IEEE Symposium on Security and Privacy*, 2023.

[4] Zhicong Huang, Wen jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In *31st USENIX Security Symposium*, pages 809–826, 2022.

[5] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *27th USENIX Security Symposium*, pages 1651–1669, 2018.

[6] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *ACM CCS*, 2020.

[7] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. DELPHI: A cryptographic inference service for neural networks. In *29th USENIX Security Symposium*, pages 2505–2522, 2020.

[8] Payman Mohassel and Peter Rindal. ABY3: A mixed protocol framework for machine learning. In *ACM CCS*, pages 35–52, 2018.

[9] Alex Ozdemir, Fraser Brown, and Riad S. Wahby. CirC: Compiler infrastructure for proof systems, software verification, and more. In *IEEE Symposium on Security and Privacy*, 2022.

[10] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-party secure inference. In *ACM CCS*, pages 325–342, 2020.